Main Search Efficiency Plan (Sciex ZT)

Cutting the 13× meta-scan overhead

feat/sciex-zt-scanning

2026-07-09

1 The problem, measured

Main Search is 56% of runtime (≈249 s / 3 files) and allocates ≈213 GB cumulatively. Per file (from the debug log):

deconv PSMs emitted (per precursor × bin × scan)	≈35,000,000
after `collapse_to_metascans` (per precursor × cycle)	≈3,200,000
reduction	**11**×
columns per PSM	61
`permute_psms_by_precursor_idx!` (Sort 1)	≈4.2 s

The waste: every feature stage runs on the full 35M rows, and only then does the collapse keep ≈1/12 of them. We compute and allocate features for ≈32M rows we immediately throw away.

2 Current order (per file)

`library_search` → **35M** PSMs (deconv, 47 fields)
`add_scan_competition_features!` [35M, *per-scan*, 147 ms]
`add_ms1_lookup_features!` [35M, *per-scan*, 1.8 s]
`permute_psms_by_precursor_idx!` [Sort 1, 4.2 s]
— `process_search_results!` —
`prepare_psm_features!` [35M, *per-PSM*, 14 cols]
`add_chromatogram_features!` [35M, *per-precursor*, 12+ cols]
`collapse_to_metascans` [Sort 2; **35M** → **3.2M**]
`train_lgbm_and_select_best`

3 Feature-dependency map (what can move)

The collapse only needs `:weight`, `:precursor_idx`, `:scan_idx` (all from deconv) — *no* computed features. So the question is which feature stages *must* run on the full 35M (they need per-scan context) vs. which are pure per-PSM / per-precursor transforms that can run on the 3.2M meta-PSMs.

Stage	Context	Movable after collapse?
`scan_competition`	per-scan (all candidates in a scan)	**No** – keep before (cheap, 147 ms)
`ms1_lookup`	per-scan (scan-contiguous MS1 cache)	No – keep before (1.8 s)
`prepare_psm_features!`	per-PSM (lookups by prec/scan idx)	**Yes** – identical on center rows
`add_chromatogram_features!`	per-precursor chromatogram, scattered to all 13 bins	**Yes** – and *cleaner* (see below)

Bonus insight: `add_chromatogram_features!` builds each precursor's chromatogram from its PSMs — currently the 13-points-per-cycle jagged trace (the same artifact we fixed in IntegrateChromatograms). Running it *after* the collapse feeds it the clean one-point-per-cycle trace, so it is both cheaper *and* more correct. (This means it is not byte-identical — it must be A/B'd, but the expected direction is neutral-to-better.)

4 Options

Option 1 — Reorder: collapse before the per-PSM features (recommended first)

Move `collapse_to_metascans` up to right after `ms1_lookup` (still ZT-gated), so `prepare_psm_features!` and `add_chromatogram_features!` run on 3.2M instead of 35M. Because the collapse already sorts by (precursor, scan), its output is precursor-grouped, so **Sort 1 (`permute`, 4.2 s) is eliminated**.

- **Savings:** ≈11× on the two big feature stages (allocation + scatter) + the 4.2 s sort. Deconv (39 s) and the two per-scan stages unchanged.
- **Risk:** moderate. `prepare_psm_features!` is byte-identical on center rows; `add_chromatogram_features!` changes value (cleaner input) ⇒ verify IDs/quant. Well-contained (moves one call, drops one sort, all ZT-gated).
- **Sub-option 1a (byte-identical):** move only `prepare_psm_features!` after the collapse and leave chromatogram features before it — smaller win, zero result change, good as a first safe landing.

Option 2 — Lightweight neighbor rows

In the deconv loop, emit full 47-field rows only for *center* bins (≈3.2M); for the ≈32M neighbor bins emit only `(precursor_idx, scan_idx, weight)` (≈12 B vs a 61-col row) into a compact side-array the collapse reads for its $\pm k$ profile.

- **Savings:** kills most of the 35M full-row materialization (the 3.5 GB deconv-output block + every 35M-scale feature column) while preserving the per-bin weights. Stacks on Option 1.
- **Risk:** higher — touches the deconv emit path (`ScoredPSMs.jl` / `process_scans_fused.jl`) and the collapse's gather.

Option 3 — Streaming / fused collapse (your "condense as we go")

Accumulate per-(precursor, cycle) weights *during* the deconv scan loop into a per-precursor accumulator, materializing only the center row. Never build the 35M table; no Sort 1, no Sort 2.

- **Savings:** the largest — eliminates the 35M table, both sorts, and the disk round-trip, and runs all features on 3.2M.
- **Risk:** highest — the deconv loop is scan-major and the per-scan stages (`scan_competition`, `ms1_lookup`) must be folded into the loop. Effectively Options 1+2 plus loop fusion. Do last.

Option 4 — Smaller k (independent, trivial)

The triangle edge weights are tiny ($t_{\pm 6} = 0.14$); k=4–5 cuts the expansion ≈40% for likely-negligible ID/quant loss. One-line env change, stacks with everything. Worth a quick A/B regardless.

On your "no additional sorts" idea

Deconv is inherently scan-major, so the sort exists to group by precursor. Avoiding it entirely means *bucketing into per-precursor accumulators as you emit* (radix-style) — which is exactly Option 3. On its own, reordering emission only removes one sort; the 11× feature waste needs the collapse-before-features move (Option 1). So the sort elimination is a *consequence* of Options 1/3, not a separate lever.

5 Recommended sequence

1. **Option 1a** (move `prepare_psm_features!` after collapse): byte-identical, drops Sort 1, first safe win. Verify IDs/quant *identical* + measure Main Search time.
2. **Option 1** (also move `add_chromatogram_features!`): A/B IDs/quant (expect neutral-to-better from the clean chromatogram); measure the full 11× feature-stage saving.
3. **Option 4** (k=5) A/B in parallel — free if neutral.
4. **Option 2**, then **Option 3** if we want to also remove the 35M materialization and the deconv-output allocation.

Verification protocol for each step: run the 3 ZT files (k=6, MBR off), compare precursor rows / protein-group rows / across-replicate median CV against the `results_3rep_iso00` baseline (73,657 / 12,228 / 8.48%), plus the Main Search stage time and cumulative alloc from the report. A step "passes" if IDs and CV are within noise and Main Search time / alloc drop.